이동형

DevOps Engineer | 경력기술서

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- **비용 최적화** - 하이브리드 전환으로 20%, GCP 마이그레이션(Fargate → GKE 노드 풀 + MSP 할인)으로 50% 절감
- **그린필드 EKS·IaC** - 신규 게임 EKS 단독 구축·런칭(플랫폼 100% Git 선언 관리, IAM 제외), 신규 컴포넌트 Terraform·Ansible IaC 100%(IAM 제외)
- **AI 워크로드 GPU 인프라** - GCP A100·L4 VM을 Terraform으로 관리(너디스타), 온프레미스 GPU에 사내 LLM 구축(팬텀)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 향상

- **파이프라인 설계·고도화** - GitHub Actions·GitLab CI·Jenkins·ArgoCD 멀티클러스터 배포 자동화, 배포 시간 67% 단축
- **CI 빌드 환경 표준화** - 공용 이미지 계층화·버전 갱신 자동화, 준비 5분 → 약 10초(약 97% 단축)

모니터링 및 관측성 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- **로그·메트릭·추적** - ELK → EFK → ECK Operator 로깅 고도화, kube-prometheus-stack 알림, OpenTelemetry + Tempo로 Metrics ↔ Traces ↔ Logs 3-Signal 연계

자동화 및 팀 역량 강화

Python·Bash 운영 자동화 도구 구축과, 파트너사 엔지니어가 스스로 운영하도록 만드는 교육·자료 표준화

- **엔지니어링 교육** - 파트너사(Delivery Partner) 엔지니어 대상 Kubernetes 6개월 실습 설계·운영(평균 10명·회당 4시간)
- **운영 자동화 도구** - 인프라 툴링 Python 전환, 차트 업그레이드 MR 자동 생성, Kubespray 업그레이드·점검 스크립트화

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

Gateway API(NGF) 전환, 공용 차트 오픈소스 공개, 업스트림 기여

- **오픈소스 공개·업스트림 기여** - Helm 차트·CRD 컨트롤러·GitHub Actions·Ansible 컬렉션 공개(ArtifactHub·GitHub Marketplace·Ansible Galaxy), apache/airflow·OpenTelemetry·prometheus-community 차트에 Gateway API HTTPRoute 지원, kubernetes-sigs(Kueue·KWOK)·Prometheus Operator·k3s·Gitea 기여
- **NGINX Gateway Fabric 업스트림 기여** - 정책 상태 조건 구현·컨트롤러 함수 11개 리팩토링·BackendTLSPolicy panic 수정으로 NGF 2.7·NGF 2.8 릴리스 공지에 연속으로 커뮤니티 기여자 등재

인프라 보안 & IAM

Keycloak Operator SSO와 Vaultwarden으로 사내 인증·시크릿 거버넌스 일원화

- **인증·시크릿** - Keycloak Operator 기반 SSO(Harbor·ArgoCD), AWS 운영 DB 평문 비밀번호 → External Secrets Operator + AWS Secrets Manager, pull secret → Sealed Secrets

주요 경력 (Professional Experience)

팬텀(콘크릿트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, 온프레미스·AWS 하이브리드 아키텍처를 설계·운영해 월 인프라 비용을 20% 절감했고, 라이브 게임의 2차 기술지원과 신규 게임의 인프라 구축을 맡고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도: 하이브리드 아키텍처 설계·구축 100% (단독) / 라이브 게임 2차 기술지원 및 운영은 외부 퍼블리셔 운영팀과 협업 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영해 트래픽 변동이 적은 비프로덕션 환경까지 클라우드 리소스를 상시 점유했고, 비용 최적화와 환경 분리가 필요했습니다.

접근 전략

- **하이브리드 아키텍처 설계**: 워크로드별 비용 및 사용 패턴을 분석해 dev·qa 환경은 온프레미스로 전환하고, 확장성과 고가용성이 필요한 review·prod 환경은 AWS(EKS·ALB)에 유지
- **트래픽 및 레지스트리 분리**: 애플리케이션 레벨에서 트래픽을 분리하고, 레지스트리도 dev·qa는 Harbor, review·prod는 ECR로 이원화
- **IaC 기반 운영 표준화**: 온프레미스와 AWS 전반에 Terraform과 Ansible을 도입해 신규 컴포넌트 IaC 100%(IAM 제외)를 달성하고, 이종 인프라 환경에서도 일관된 형상 관리 체계 수립

성과

- 워크로드 분석을 근거로 한 하이브리드 재배치로 월 인프라 비용을 20% 절감하고, 절감분은 신규 프로젝트에 재투자

프로젝트 2: 게임 애플리케이션 CI/CD 파이프라인 구축 및 CI 빌드 환경 표준화 (App GitOps)

기여도 100% (파이프라인·CI 표준화 전 과정 단독 설계·구축) 2024.12 ~ 현재 (운영 중)

게임 애플리케이션 배포를 GitOps로 자동화하고, 사내 저장소가 공유하는 CI 도구 이미지와 버전 관리를 표준화했습니다.

2-1. 게임 애플리케이션 배포 CI/CD 파이프라인 (App GitOps, 2024.12 ~ 현재)

문제

개발자가 수동으로 빌드 후 서버에 직접 배포하는 방식이라 배포당 평균 30분이 소요되었고 휴먼 에러로 인한 서비스 장애가 잦았습니다.

접근 전략

- **GitOps 기반 지속 배포(CD) 파이프라인 구축**: GitLab CI와 ArgoCD를 연계해 Git Push부터 빌드·테스트·배포를 자동화하고, Git과 어긋난 클러스터 상태는 ArgoCD app-of-apps 구조로 자동 복구(Self-healing). dev·qa·review·prod 4개 환경을 단일 파이프라인으로 통합
- **최소 권한 기반의 안전한 이미지 빌드 환경 구축**: Privileged 모드가 필요한 Docker-in-Docker 대신 데몬 없이 빌드하는 Kaniko로 전환해, 빌드 잡이 침해돼도 호스트까지 장악될 위험을 낮춤

성과

- 배포 시간 67% 단축 (30분 → 10분) - 수동 단계를 파이프라인으로 일원화해 휴먼 에러라는 배포 장애의 근본 원인을 제거
- 데이터 배포 소스 페치 시간 약 78% 단축 (약 139초 → 약 30초, shallow + blobless fetch)
- 핵심 서비스 저장소당 주 평균 100회 배포, 기획 데이터(MasterData)가 바뀌면 이를 쓰는 하위 저장소의 배포 파이프라인을 자동 트리거

2-2. 공용 CI 도구 이미지 계층 구조 및 자동 버전 사이클 (2025.09 ~ 현재)

문제

CI 잡마다 필수 도구(helm·helmfile·kubectl·crane·aws-cli)를 설치해 빌드 환경 준비에 평균 5분이 걸렸고, 도구 버전이 저장소별로 흩어져 새 버전을 반영하려면 전부 고쳐야 했습니다.

접근 전략

- **2계층 공용 도구 이미지**: 외부 레지스트리 이미지를 멀티 아키텍처(amd64·arm64) 매니페스트 그대로 사내 Harbor에 미러링하고(Layer 1), 그 위에 도구를 미리 설치한 공용 이미지 6종(Layer 2)을 만들어 저장소는 이미지 태그만 지정
- **공용 CI 템플릿과 분산 캐시**: 저장소마다 복사하던 CI 잡을 `include`로 참조하는 공용 템플릿으로 모으고, 온프레미스에 S3 호환 스토리지가 없어 오류만 내던 캐시는 직접 만든 오픈소스 정적 파일 서버에 GitLab Runner용 S3 캐시 모드를 추가해 해결(10개 저장소 채택)
- **자동 버전 사이클 구축**: 한 번 실행하면 업스트림 신규 버전 감지부터 미러링·리빌드 MR 생성까지 이어지고, 중앙 버전 변수만 바꾸면 하위 저장소로 전파되며, 머지 승인은 사람이 맡음. helmfile 1.5.1이 helm 3.18.6 이상을 요구해 고정 버전 3.16.4에서 빌드가 멈춘 뒤로는 같은 minor 안의 patch만 자동 상향

성과

- **CI 프로비저닝 시간 약 97% 단축 (5분 → 약 10초)**: 사전 빌드된 공용 이미지로 빌드 지연을 해소하고 사내 저장소 전반의 도구 버전 일관성 100% 달성
- 업그레이드 대상을 위험도 티어(무상태·플랫폼·데이터)로 나눠 한 목록에서 파이프라인을 만들고, 머지·클러스터 적용은 사람이 승인하며 데이터 티어는 사전 점검

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

중앙 로깅을 3단계로 고도화하고, 리텐션 분석으로 확장했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

서버와 컨테이너 로그가 흩어져 있어 Pod가 재생성되면 로그가 사라졌고, 장애가 나면 노드와 Pod에 하나씩 접속해 로그를 찾아야 했습니다.

접근 전략

- **EFK 경량화와 ECK Operator 전환**: JVM 기반 Logstash의 메모리 부하를 줄이려 수집은 Fluent-bit DaemonSet, 파싱은 Fluentd로 나누고, 2년 넘게 정체된 공식 Helm 차트 대신 ECK Operator의 CRD로 수명주기를 선언적으로 관리
- **클라우드(EKS) 확장 및 정적 키 없는 자동화**: 같은 스택을 템플릿화해 EKS에 배포하고, ILM과 S3 스냅샷 백업 경로에 IRSA를 적용해 정적 자격증명 제거

성과

- Pod가 사라져도 로그가 남고, Pod마다 접속하던 로그 확인을 Kibana 한 화면의 시간순 추적으로 대체
- TLS 인증서 갱신·계정 프로비저닝·노드 교체를 CRD 선언으로 자동화하고, Elastic Stack 메이저 업그레이드 완료 (8.x → 9.x)

3-2. Fluent-bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 현재)

SQLite 오프셋 DB를 노드 로컬에 두어 수집기 재시작에도 중복 수집 0건을 유지하고, 수집만 멈춘 상태를 알림으로 잡았습니다. Elasticsearch 롤링 재시작 중 Fluentd가 실패한 로그를 재시도 없이 버리던 원인을 플러그인 코드에서 찾아 묶음 단위 재전송으로 고쳤습니다.

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

Elasticsearch Transform(5분 주기)으로 인게임 이벤트를 사용자당 1행으로 집계해, 별도 SaaS 없이 Kibana로 D+1~D+30 리텐션과 코호트를 분석합니다.

프로젝트 4: 개발 생산성 도구 구축 및 셀프서비스 운영 콘솔

기여도 100% (전체 시스템 설계 및 개발)　2025.02 ~ 현재

개발팀의 반복 수동 작업을 줄이는 사내 도구 6종을 만들고, 셀프서비스 콘솔로 묶었습니다.

4-1. APK 배포 자동화: Slack 알림 봇 · 정적 파일 서버 (2025.02 ~ 현재)

Jenkins APK 빌드 완료 시 Slack으로 다운로드 QR을 보내는 봇으로 QA 전달 시간을 90% 단축했고, 기존 파일 서버는 직접 개발해 오픈소스로 공개한 Go 정적 파일 서버로 교체했습니다.

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab push 때 디자인 문서를 Google Drive로 자동 동기화해 업로드 시간을 10분에서 2분으로 줄였습니다.

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI로 사내 LLM 서비스를 4주 만에 구축하고, 코딩 모델을 GPU 메모리에 상주시켜 개발팀 공용으로 운영했습니다.

4-4. Git 저장소 미러링 도구 개발 및 Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 현재)

CodeCommit·GitLab·GitHub 간 수동 동기화의 누락과 지연을 없애려 Go 기반 미러링 도구 git-bridge를 개발해 오픈소스로 공개했고, 실패 재시도·Slack 알림·종료 시 드레인(실측 중앙값 2.9초)을 갖춰 사내 신·구 GitLab을 잇는 상시 동기화 경로로 운영합니다.

4-5. 사내 셀프서비스 운영 콘솔 플랫폼 구축 (2026.07 ~ 현재)

문제

계정 생성·삭제가 인프라 담당자만 실행할 수 있는 스크립트에 묶여 모든 요청이 한 사람을 거쳤고, 흩어진 계정 시스템 10개의 권한을 감사하기 어려웠습니다.

접근 전략

- **단일 OIDC 인증 게이트웨이와 통합 포털**: 게이트웨이 뒤에 사내 웹 도구와 운영 서비스 5종(계정·Keycloak·Harbor·Elasticsearch·VPN)을 포함한 7개 애플리케이션을 모으고, 계정 시스템 10개를 연동해 모든 계정 변경의 작업 주체를 기록

트러블슈팅

- **CI 빌드와 GitOps 동기화 레이스 컨디션으로 생긴 다운타임**: 이미지 빌드가 끝나기 전에 ArgoCD가 새 태그를 동기화해 `ImagePullBackOff`가 났고, RWO 볼륨 때문에 `Recreate` 전략을 쓰는 서비스(8개 중 6개)는 전면 중단됨.
  동기화 전에 이미지를 실제로 pull해 보는 PreSync 훅으로 해결(Harbor 운영 서비스 기준 중단 약 180초 → 정상 교체 약 20초)

성과

- **운영 병목 해소 및 셀프서비스 전환**: 6주간 단독 개발해, 한 사람을 거치던 계정·권한 요청을 각 팀이 자기 권한 안에서 직접 처리

프로젝트 5: Kubernetes 클러스터 · 사내 시스템 운영 고도화

기여도 100% (단독 설계 및 표준 절차를 자동화 스크립트로 코드화해 팀 내 재사용)　2025.12 ~ 현재

클러스터와 사내 시스템의 운영 전반을 정비했습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack에 심각도별 알림과 상위 알림 발생 시 하위 알림 억제를 적용해 중복 통보를 막았고, CRD 삭제로 Prometheus가 멈춘 줄 아무도 몰랐던 일을 계기로 클러스터 밖 Bastion의 watchdog이 5분마다 모니터링 스택을 확인하게 했습니다.

5-2. Kubernetes 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespray 업그레이드와 사후 점검을 스크립트화했고, 2026.11 온프레미스 1.34 → 1.36 업그레이드에서는 DB 노드의 drain 유예만 조정해 DB 중단을 약 7분에서 약 4분으로 줄이고, etcd 3.6에서 확인 없이 통과만 하던 백업 검증을 찾아 고쳤습니다.

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

여러 ingress-nginx 인스턴스를 NGF 단일 컨트롤플레인으로 통합하며 MetalLB IP를 유지해 DNS·방화벽 변경 없이 클래스당 30~60초에 전환했습니다. 이후 NGF가 HTTPRoute 타임아웃을 받아들이고도 기본 60초에 요청을 끊던 것을 찾아 값을 ProxySettingsPolicy로 옮기고 공용 차트에서 막았습니다.

5-4. 스토리지 성능 최적화: etcd / DB SSD 마이그레이션 및 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

빌드 중 GitLab Runner의 디스크 I/O로 HDD의 etcd와 게임 DB가 느려지던 병목을, 컨트롤플레인·DB 워커를 SSD로 옮기고 빌드를 별도 물리 서버의 VM으로 분리해 해소했습니다. 게임 DB COMMIT 지연이 9~14초에서 ms 단위로 돌아왔고 빌드 중 서비스 저하는 0건이 되었습니다.

5-5. Kubernetes 인프라 배포 자동화 및 멀티클러스터 GitOps(ArgoCD) 전환 (2026.03 ~ 현재)

문제

Shell/awk 인프라 모노레포는 테스트할 수 없었고, CI가 `helmfile apply`로 미는 push 방식이라 드리프트를 잡지 못했으며 멀티클러스터로 넓히기 어려웠습니다.

접근 전략

- **Python 툴링과 Wave 배포·업그레이드 자동화**: 외부 패키지 없는 Python 모듈과 단위 테스트로 리팩토링하고, 6단계 Wave 배포 순서와 차트 새 버전의 Values diff·호환성 결과를 담은 MR 자동 생성 파이프라인 도입
- **ArgoCD pull-GitOps 전환**: ApplicationSet으로 온프레미스·AWS 배포를 자동화하고, 기존 Helm 릴리스는 리소스를 다시 만들지 않고 ArgoCD 관리로 무중단 이관
- **인프라 저장소를 역할별 3개로 분리**: 'ArgoCD를 띄우거나 복구하는 데 필요한가'를 기준으로 부트스트랩용 Helmfile 컴포넌트, ArgoCD가 배포하는 컴포넌트, 호스트·VM 운영을 나누고, Git 소스 교체 중에는 리소스 삭제 방지 옵션을 임시로 켜 무중단 이관
- **GitOps 거버넌스 및 안전장치 마련**: 최상위 계층의 연쇄 삭제를 막고 AppProject 허용 목록으로 장애 영향 범위를 제한. 이름 와일드카드 때문에 테스트 프로젝트가 프로덕션 클러스터에도 배포될 수 있던 허점을 찾아 서버 주소로만 허용하도록 고치고 검증 스크립트로 재발 방지

성과

- 플랫폼 릴리스는 온프레미스·AWS 클러스터마다 둔 ArgoCD가 같은 Git 저장소를 따라 배포하고, 버전 확인·배포 순서를 챙기던 수작업은 없앰

5-6. 백업·복구(DR) 체계 구축과 백업 검증 (2026.08 ~ 2026.09)

사내 시스템 8곳과 NAS 경로 2곳의 백업·복구를 갖추고, 검증 기준을 '파일이 끝까지 온전한가'로 바꿔 DB 덤프가 잘렸는데도 정상 종료돼 복원 때 실패했을 백업을 찾아 고쳤습니다.

5-7. 사내 GitLab 이원화와 구 GitLab 21단계 메이저 업그레이드 (2024.10 신규 구축 / 2026.08 ~ 2026.09 업그레이드)

입사 직후 새 GitLab을 베어메탈에 구축하고 구 GitLab은 직접 개발한 git-bridge로 동기화하며, 오래 정체된 그 버전을 공식 Upgrade Path대로 6일간 21단계로 올렸습니다.
다운타임은 단계당 3~4분(첫 단계 약 6분 30초)이었고, 야간 동기화 1회 전송량은 123G에서 39G로 68% 줄었습니다.

프로젝트 6: 인프라 보안 및 거버넌스 체계 구축

기여도 100% (설계·배포·SSO 연동) 2026.04 ~ 현재

로그인, 시크릿, 원격 접속을 하나의 기준으로 정리했습니다.

6-1. 중앙 인증(SSO/OIDC) 통합 (Keycloak Operator, 2026.04 ~ 현재)

Harbor·ArgoCD가 각자 GitLab OIDC에 연결하던 로그인을 Keycloak Operator 기반 중앙 IdP로 통합하고, GitLab 그룹이 Keycloak을 거쳐 ArgoCD 역할로 이어지게 해 권한을 한곳에서 관리합니다.

6-2. 시크릿 거버넌스 수립: Vaultwarden · GitOps Secrets · 토큰 감사 (2026.04 ~ 현재)

흩어져 있던 사내 공용 비밀번호를 SSO 뒤의 Vaultwarden으로 모으고, Git에 평문으로 커밋되던 AWS 운영 DB 접속 정보는 External Secrets Operator와 AWS Secrets Manager로, Harbor pull secret은 Sealed Secrets로 옮겼습니다.
토큰과 미러 상태는 매주 점검합니다.

6-3. 사내망 보안 원격 접속 환경 구축 및 이중화 (OpenVPN PKI, 2026.07 / 이중화 2026.10)

동시 접속 50명 기준의 PKI 인증서 방식 VPN 게이트웨이를 구축하고 사용자별 인증서 발급·폐기를 스크립트로 자동화했습니다.
이후 서버 한 대에 몰려 있던 VPN과 AWS WireGuard 터널을 keepalived active-standby로 이중화하고(장애 감지 3~4초), 전환 훈련에서 AWS 경로 끊김 1.5초 이하를 확인했습니다.

프로젝트 7: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (단독 설계·구축·운영) 2026.06 ~ 2026.11 (이후 프로젝트 8로 이관)

신규 퍼블리싱 게임의 EKS 프로덕션 환경을 단독 구축하고, 추적·Canary·CDN 보안을 더해 소프트 런칭한 뒤 SLI/SLO를 붙였습니다.

7-1. 그린필드 EKS 프로덕션 인프라 구축 및 실트래픽 최적화 (2026.06 ~ 2026.11)

문제
제품마다 AWS 계정을 따로 두는 원칙에 따라 신규 외부 퍼블리싱 게임은 새 계정에서 해외 상용 런칭을 해야 했습니다. 그래서 기존 클러스터에 올리지 않고, 그 계정에 고가용성 EKS 프로덕션 환경을 처음부터 구축했습니다.

접근 전략
- **GitOps 관리와 워크로드별 노드 분리**: 부트스트랩 계층(Karpenter·LBC·ArgoCD)은 helmfile, 나머지 플랫폼 컴포넌트는 ArgoCD로 관리하고, Karpenter 노드 풀은 모두 ARM으로 두되 게임 서비스는 세션 중 노드가 회수되지 않게 On-Demand에, CI 러너는 여러 인스턴스 패밀리의 Spot에 분산
- **네트워킹·인증 자동화**: AWS Load Balancer Controller·ExternalDNS·wildcard ACM 인증서로 Ingress만 선언하면 ALB 규칙·DNS·HTTPS가 자동으로 이어지게 하고, 모든 ServiceAccount에 Pod Identity/IRSA를, 시크릿에 External Secrets Operator를 적용해 워크로드의 정적 IAM 키 제거

트러블슈팅
- **빈 노드가 회수되지 않던 문제**: 실사용 101MiB인 서비스의 메모리 request 2GiB를 512MiB로 낮춰 92%와 35%로 벌어졌던 노드 점유를 고르게 맞추고 Consolidation이 동작하게 함
- **노드 교체 시 Pod 쏠림과 Consolidation 루프**: AMI 교체 때 한 서비스의 Pod 2개가 새 노드 한 대에 몰려 17초간 Degraded된 문제를 배포 버전 단위의 노드 간 분산 제약으로 해결.
  같은 제약을 다른 서비스에 걸자 사용률 기반 Consolidation이 약 7분 30초마다 노드를 16번 다시 띄워, 운영 중인 Pod를 계속 축출하느니 빈 노드만 정리하는 정책으로 되돌림

성과
- 신규 게임 전용 AWS 계정에 EKS 프로덕션 환경을 처음부터 구축하고, 플랫폼 매니페스트 100%를 Git으로 선언 관리(IAM 제외)
- Bastion SSH 키(배포를 Jenkins → S3 → CloudFront·SSM 경유로 전환)와 클러스터 워크로드의 정적 IAM 키를 제거하고, CI 에이전트에 남은 AWS 키는 역할 전환 권한만 갖도록 제한

7-2. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo, 2026.07 ~ 현재)

문제
EFK 로깅과 Prometheus 메트릭만으로는 마이크로서비스 사이의 호출 흐름, 지연이 생기는 구간, 에러가 번지는 범위를 알기 어려웠습니다.

접근 전략
- **추적 스택과 자동 계측**: OpenTelemetry Collector와 Grafana Tempo(S3)를 구축하고 OTel Operator 자동 계측으로 코드 수정 없이 SDK 주입
- **메트릭 추출 지점과 저장 지점을 분리**: RED 메트릭은 샘플링 전 100% Span에서 뽑아 집계를 정확히 유지하고, 저장은 Tail Sampling(에러 전체 + 500ms 초과 요청 + 정상 요청 10%)만 남겨 비용을 줄임

성과
- Metrics·Traces·Logs를 Grafana 한 화면에서 오가게 연결해 서비스 간 병목을 가시화하고, 트레이스에서 그 요청의 로그만 trace_id로 바로 열리게 함
- Collector replica마다 커넥션이 고정돼 메트릭 시리즈 1,650 : 541로 한 replica에 쏠려 있던 부하를, Trace ID 기반 2계층 라우팅으로 두 샘플링 인스턴스가 초당 Span 4.09 : 5.18로 고르게 나눠 받게 함

7-3. Argo Rollouts 기반 Canary 무중단 배포 전환 (Blue-Green → Canary, 2026.07 ~ 현재)

Blue-Green 배포는 promote 때 Target Group을 통째로 바꿔, 새 Pod가 등록되기 전에 기존 Pod가 빠지면 요청이 503으로 실패했습니다. 같은 Target Group 안에서 새 Pod가 준비된 뒤 구버전을 내리는 Canary로 바꾸고 ALB 등록 해제 지연(30초) < preStop(35초) < 종료 유예(60초)로 종료 시점을 맞춰, 공개 트래픽을 받는 메인 서비스의 하위 호환 패치를 무중단으로 배포합니다.
두 버전이 함께 떠 있으면 안 되는 변경은 정기 점검 때 반영합니다.

7-4. 소프트 런칭 운영 (2026.07 ~ 현재)

성과

- 소프트 런칭 트래픽을 Canary 배포·3-Signal 관측 위에서 처리하고 오토스케일링(HPA·Karpenter)까지 갖춤
- 소프트 런칭 2주간의 실트래픽을 근거로 Pod requests·limits를 다시 잡아, requests가 과도해 HPA가 스케일아웃하지 못하던 구조를 바로잡고 requests를 CPU 48%·메모리 71% 줄임

7-5. SLI/SLO·에러 버짓 정의 (2026.08)

샘플링 전 전체 Span에서 뽑은 요청 지연시간으로 SLI 기록 규칙과 에러 버짓 소진을 비교하는 SLI/SLO 대시보드를 만들고, 늘 통과해 의미가 없는 목표('p95 2초 미만' 등)는 실측 분포로 걸러냈습니다.
게임 서버가 예외에도 HTTP 200을 돌려줘 약 790만 건의 요청 Span 중 에러가 0건이어서, 장애 중에도 100%로 보일 가용성 SLI는 애플리케이션이 에러를 에러로 돌려주기 전까지 보류했습니다.

7-6. 게임 CDN 엣지 보안: AWS WAFv2 적용 (2026.07 ~ 2026.09)

런칭을 앞두고 WAF 없이 노출돼 있던 게임 CloudFront 앞단에 WAFv2를 도입했습니다. AWS 관리형 룰 그룹 4종을 Count 모드로 실트래픽에서 먼저 관측해 정상 유저를 막지 않았고, 이후 내부 API 경로는 운영자 허용 IP 밖의 요청을 차단했습니다.
CGNAT 뒤의 유저까지 막을 수 있는 Rate-based 룰은 데이터가 쌓일 때까지 보류했습니다.

프로젝트 8: EKS Auto Mode 클러스터로 프로덕션 이관과 Site-to-Site VPN(WireGuard) 게이트웨이 구축

기여도: 데이터 계층·IAM·CDN·VPN·관측·이관 단독 설계·실행 2026.08 ~ 2026.11 (프로덕션 전환·구 클러스터 정리 완료)

문제

퍼블리셔 계정과 VPC 피어링이 필요해, 퍼블리셔 표준 규격의 새 VPC·EKS Auto Mode 클러스터로 데이터 계층과 워크로드를 옮겨야 했습니다.

접근 전략

- **EKS Auto Mode로 컴퓨트 계층을 관리형으로 전환**: 노드 수명주기, Karpenter 운영에 필요한 AWS 리소스, 내장 애드온을 AWS에 맡기고 코드에는 NodePool 선언만 남김.
  NodePool·IngressClass는 클러스터 재구축 때 ArgoCD가 Pending에 멈추지 않도록 ArgoCD보다 먼저 helmfile로 배포
- **Site-to-Site WireGuard VPN 게이트웨이 구축**: 퍼블릭 ALB를 늘리면 환경마다 호스트명·인증서·DNS 레코드가 함께 늘어나므로 터널로 도메인·인증서를 하나로 유지하고, VPC 라우트는 사무실 라우터 한 곳에서 관리
- **테스트 계정 EKS 클러스터를 ECS Fargate로 대체**: 클러스터 전체가 CPU 50m·메모리 128Mi를 요청하는 프록시 Pod 하나만 돌리고 있어, 내부 ALB 뒤 Fargate 서비스로 옮기고 클러스터를 폐기. 비용이 같은 EC2는 롤링 배포를 쓸 수 없어 제외
- **CloudWatch 알림 이관과 감시 공백 해소**: 전환 때 알림을 새 스택으로 옮겨 새 Aurora·Valkey 등 데이터 계층이 알림 없이 투입되지 않게 하고, 알림 Lambda 오류는 Lambda를 우회한 이메일로 받고, AWS Prometheus는 온프레미스 dead-man's switch로 감시

트러블슈팅

- **Auto Mode가 ALB 그룹 어노테이션을 무시하는 문제**: 내장 컨트롤러는 `group.name`을 읽지 않고 `IngressClassParams`로만 그룹을 인식해, 그대로 두었다면 service 그룹 Ingress 5개 중 4개가 ALB를 할당받지 못할 상황이었음.
  ALB 그룹마다 IngressClass·IngressClassParams를 선언해 해결
- **프로덕션 전환 중 CDN 리스너 룰 생성 실패**: Auto Mode 관리형 정책이 자체 태그 키만 허용해, 공용 차트가 붙인 사용자 정의 ALB 태그 때문에 리스너 룰 생성이 403으로 실패함. 태그를 비우고 이미 붙은 태그를 직접 제거해 복구

성과

- 컴퓨트 계층의 Terraform 관리 대상을 83% 축소(102개 → 17개)하고, 노드 풀 정책만 NodePool로 직접 관리
- IAM·CDN·인증서·ECR 등 계정 단위 자산 15개를 삭제 0건으로 state만 옮겨, ARN과 주소가 그대로라 CI·매니페스트 수정 0건
- 심사 환경을 먼저 옮겨 검증한 뒤 정기 점검 시간에 프로덕션을 전환해, DB 스키마를 53초 만에 이전하고(리허설 약 1분) 36개 테이블 전부의 행 수·체크섬 일치를 확인. 같은 날 구 클러스터를 정리해 병행 운영 종료
- 테스트 계정 클러스터를 ECS Fargate로 대체해 월 비용을 약 250달러에서 약 28달러로 약 89% 절감(2026.10 Cost Explorer 실측 → 2026.11 정가 기준 산정)

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며 AWS → GCP 대규모 클라우드 마이그레이션에서 인프라 부문을 총괄했습니다.
비용 최적화와 글로벌 네트워크 성능 확보를 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스는 GitLab, 프로덕션은 GitHub로 이원화해 관리했고, 정체돼 있던 베어메탈 GitLab을 서버 교체 없이 공식 경로를 따라 다단계(약 12 hop)로 업그레이드했습니다.
또한 MongoDB·CloudSQL·GA·Dune 등 멀티 소스 데이터를 BigQuery로 모아 Google Sheets까지 자동 갱신하는 데이터 수집 파이프라인을 구축해, 분석가가 매일 2~3시간 들이던 수동 수집을 자동 적재로 대체했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30%는 서버 개발팀의 애플리케이션 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 계속 올랐고, 특히 상시 가동되는 게임 워크로드를 Fargate로 돌리며 발생하는 Pod 단위 과금이 큰 비중이었습니다. 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

- **3단계 순차 이전** - 1단계 개발 환경 GCP 구축·검증, 2단계 QA, 3단계 프로덕션
- **네트워크·인증** - Shared VPC(Host Project / Service Project 분리) 설계, GitHub Actions의 GCP 인증은 Workload Identity Federation으로 전환해 서비스 계정 키 발급·순환을 OIDC 단기 토큰으로 대체.
  DNS는 서브도메인 위임(Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증하고, 최종 NS 변경만 점검 공지 후 전환
- **컴퓨트 재배치** - AWS Fargate 워크로드를 GKE 표준 노드 풀로 이전. 상시 워크로드는 Pod 단위 과금이 VM 단위보다 불리해 동일 과금 구조인 GKE Autopilot 제외
- **스토리지 제약 우회·WAF** - Filestore SSD 최소 2.5TB 제약은 Compute Engine에 pd-balanced 1TB 디스크를 붙인 NFS 서버 직접 구축으로 우회, Cloud Armor로 리전 차단·IP 화이트리스트 웹 방화벽(WAF) 정책 구성
- **IaC·GitOps 파이프라인** - Terraform으로 GCP 인프라 IaC화(IAM 제외 100%), 이전 후 GitLab CI + ArgoCD GitOps CI/CD 파이프라인 구축, Helm 차트로 환경별 설정 표준화

트러블슈팅

- AWS ALB는 대상이 모두 헬스체크에 실패하면 막지 않고 전체에 트래픽을 보내지만 GCP 로드밸런서는 차단해, AWS에서는 드러나지 않던 헬스체크 실패가 GCP로 옮기자마자 트래픽 단절로 나타남. GKE가 Pod를 NEG로 직접 붙이는 구조에 맞춰 헬스체크와 백엔드 설정을 다시 잡아 해결
- GKE Ingress의 BackendConfig에 지정한 헬스체크 경로가 응답하지 않아 503이 발생. 그 경로가 항상 응답하게 하고 ManagedCertificate·FrontendConfig 설정과 맞춰 외부 트래픽을 정상화
- Cloud CDN으로 옮긴 뒤 HTTP → HTTPS 리다이렉트가 빠져 게임 클라이언트 파일 다운로드가 실패. HTTPS로 넘겨 주기만 하는 HTTP용 URL Map을 따로 두고, 80 포트 forwarding rule을 그쪽으로 연결해 해결

성과

- AWS → GCP 이전, Fargate → GKE 표준 노드 풀 전환, MSP 리셀러 할인을 합쳐 클라우드 비용 50% 절감
- 3단계로 전체 서비스를 이전 완료. AWS를 그대로 운영하면서 GCP에 같은 구성을 새로 만들어 두고, 최종 DNS 전환만 2시간 내외 점검 공지 후 실행해 다운타임을 줄임
- IAM을 제외한 GCP 인프라를 모두 Terraform으로 관리하고, 배포 이력은 Git 커밋으로 모두 추적
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담과 키 유출 리스크 제거

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임·블록체인 데이터를 수동으로 수집하다 보니 분석가가 매일 2~3시간을 소비했고, 휴먼 에러로 인한 데이터 누락도 빈번했습니다.

접근 전략

- **멀티 소스 BigQuery 적재** - MongoDB는 Dataflow Flex Template ETL, CloudSQL은 BigQuery 직접 connection, Google Analytics · Dune은 각각의 API(Dune은 별도 API Key)로 적재, 공통 스키마로 정규화
- **분석가용 Google Sheets 자동 갱신** - Python Cloud Function이 BigQuery 쿼리 결과를 Google Sheets API로 적재, Cloud Scheduler가 Daily(전일)·Monthly(전월) 두 주기로 자동 트리거
- **전체 인프라 Terraform IaC** - BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM · Service Account를 코드로 관리

트러블슈팅

- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 요청 한도로 결과 누락·타임아웃 발생. 실행 상태 폴링과 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 레코드가 BigQuery에 중복 적재되는 문제 발생. 중복 제거 Cloud Function에 MERGE 및 ROW_NUMBER() 기반 정합성 로직을 적용해 적재를 멱등화
- BigQuery 결과를 Google Sheets API로 적재할 때 호출이 몰리면 분당 요청 한도(429)에 걸리고, 결과가 크면 한 번에 쓰지 못해 갱신이 실패. 결과를 1,000행씩 나눠 날짜별 시트에 쓰고, 호출 사이에 간격을 두며 한도를 넘으면 지수 백오프로 재시도하게 해 한도 안에서 동작하게 함

성과

- 분석가가 매일 2~3시간 들이던 수동 수집을 자동 적재로 대체하고, 수작업 입력 실수로 생기던 누락을 없앰
- 전체 인프라를 Terraform으로 관리해 같은 파이프라인을 다른 GCP 프로젝트 3곳에 빠르게 재구축하고, 변경 이력은 Git으로 모두 추적

프로젝트 3: AI 워크로드용 GPU 인프라 표준화 및 확장 (Windows → Linux · 세팅 자동화 · 사내 / GCP / 외부 IDC)

기여도 100% (인프라 설계·구축·운영) 6개월 (2024.02 ~ 2024.07)

문제

사내 GPU 서버가 Windows로 운영돼 이미지 생성 모델(Stable Diffusion 계열)을 돌릴 때마다 엔지니어가 실행 환경을 직접 세팅해야 했습니다. 드라이버·CUDA·프레임워크 조합을 손으로 맞추다 보니 같은 환경을 재현하기 어려웠고, 장비는 있는데 쓰기까지 시간이 걸렸습니다.

접근 전략

- **Windows → Linux 표준화** - 사내 GPU 서버를 Ubuntu 22.04로 전환하고, 세팅 스크립트 하나로 드라이버 스택(nvidia-driver-550 · CUDA 11.8 · cuDNN 8.6) 설치·동작 확인, 공유 모델 저장소(NFS) 마운트, GPU별 Stable Diffusion WebUI 기동까지 처리해 어느 서버에서나 같은 환경을 재현. LoRA 학습용 kohya_ss와 ComfyUI도 같은 구성에 배치
- **GCP 클라우드 확장** - A100 40GB와 L4 GPU VM을 Terraform으로 관리하며 수요에 맞춰 구성을 바꿔 운영하고, 최대 A100 4장과 L4 8장까지 확장. 리전·존별 GPU 가용성 확인과 쿼터 상향을 거쳐 앞선 마이그레이션의 Shared VPC 서브넷에 연결해 네트워크 표준 유지
- **외부 IDC 20대 증설·워크로드별 카드 분리** - 같은 세팅 스크립트로 GPU 서버 20대 구축.
  학습은 VRAM이 큰 카드가 필요하고 추론은 요청마다 독립적이라 GPU 수만큼 처리량이 늘어, 학습·대용량 작업은 GCP A100, 대량 추론은 IDC의 소비자용 RTX 40 시리즈로 나눠 배치

성과

- Windows에서 손으로 하던 세팅을 Linux와 세팅 스크립트로 대체하고, 그 스크립트를 사내·GCP·외부 IDC GPU 서버 전체의 표준 구축 절차로 사용
- GCP GPU VM을 Terraform으로 관리해 구성 변경과 재구축을 코드로 추적하고, 기존 GCP IaC 저장소에 그대로 편입
- GPU 자원을 사내 1대에서 사내 10대, GCP A100·L4, 외부 IDC 20대로 확장하고, 학습과 추론에 맞춰 카드 등급을 나눠 배치

아이오차드

2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 프라이빗 클라우드 인프라를 설계·구축하고, 파트너사 엔지니어 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 서로 달라, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

- **HA 클러스터 설계** - 고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA 구성으로 설계하고 OpenStack으로 가상머신 관리 환경 구축
- **통합 스토리지** - Ceph로 블록·파일·오브젝트 스토리지 통합 제공

트러블슈팅

- **계획 정지·기동 절차** - 고객사 정전 작업은 전원을 내렸다 올리는 일이라 무중단이 불가능해, 데이터가 깨지지 않게 멈추고 문제없이 다시 올라오게 하는 절차를 만듦. VM부터 멈추고 DB·메시지 큐를 거쳐 스토리지를 마지막에 끄며, 다시 올릴 때는 역순으로 진행.
  DB 클러스터(MariaDB Galera)는 한 대씩 줄여 최신 데이터가 남은 노드부터 다시 올리고, 스토리지(Ceph)는 노드가 꺼져도 장애로 보고 데이터 복구를 시작하지 않도록 미리 설정. 현장에서 풀지 못한 기동 단계 문제(메시지 큐 노드가 클러스터에 다시 붙지 않음, 볼륨 생성 실패)는 2차 기술지원으로 복구

성과

- 5개 고객사에 HA 구성 PaaS 솔루션 구축·납품 후 안정 운영
- Ansible로 서버 프로비저닝과 구성 관리를 자동화해, 고객사마다 같은 플레이북으로 서버를 구성하고 배포 시간을 줄임
- 정전 작업용 정지·기동 절차서를 현장 엔지니어에게 넘기고, 기동 중 생긴 문제의 조치사항도 같은 절차서에 반영

프로젝트 2: 파트너사 엔지니어 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

파트너사 엔지니어들이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족해, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상됐습니다.

접근 전략

- **단계별 커리큘럼** - Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고 실습 환경을 구성해 실무 중심으로 운영
- **PaaS 스택 병행 교육** - OpenStack·Ceph 스토리지 운영 교육을 병행해 전체 PaaS 스택 운영 역량 확보

성과

- 6개월간 월 1회, 회당 4시간·평균 10명 규모로 교육하고 운영 가이드를 전달해 파트너사 엔지니어들이 Kubernetes를 스스로 운영하게 하고 교육 후 기술 문의를 줄였으며, 표준화한 교육 자료는 이후 다른 고객사 교육에도 재사용